

Actividad: Desarrollo de un Lexer para un Transpilador de Pseudocódigo a Python (Turtle)

Objetivo general

Desarrollar un **analizador léxico (lexer)** para un lenguaje de pseudocódigo en español, con el propósito de construir un **transpilador** que genere programas equivalentes en **Python utilizando la biblioteca gráfica turtle**.

Contexto técnico

La arquitectura base para el proceso de análisis léxico es:

- Una clase `Scanner` que procesa la cadena fuente.
- Una clase `Token` que representa las unidades léxicas.
- Una enumeración `TokenType` que define los tipos de tokens del lenguaje.
- Un punto de entrada `main` que coordina el flujo de la ejecución.

El sistema debe reconocer elementos como:

- Comandos (`COMANDO`)
- Números, cadenas y booleanos (`NUMERO` , `CADENA` , `BOOLEANO`)
- Identificadores (`ID`)
- Operadores aritméticos, lógicos y de comparación
- Estructuras de control (`SI` , `MIENTRAS` , `PARA` , etc.)
- Elementos gráficos (`COLOR` , `DIRECCION`)

Estos tipos están claramente definidos en `TokenType` y organizados por categorías léxicas en un archivo [TokenTypes.py](#)

Los tokens generados estarán encapsulados por la clase `Token` , la cual incluye información de tipo, valor, línea, columna y posición, además de utilidades de clasificación en un archivo [Token.py](#)

Descripción de la tarea

Parte 1: Implementación del Lexer (Scanner)

Implementa la clase `Scanner.py`

1. Realiza la implementación completa del analizador léxico para que sea capaz de:
 - Leer el texto fuente carácter por carácter.
 - Reconocer patrones léxicos (palabras clave, números, operadores, etc.).
 - Generar una lista de objetos `Token` .
 2. El lexer debe:
 - Identificar correctamente palabras clave como: `si` , `sino` , `mientras` , `para` , etc.
 - Diferenciar entre identificadores (`ID`) y comandos (`COMANDO`).
 - Manejar literales: números, cadenas y booleanos.
 - Ignorar comentarios (`COMENTARIO`) y manejar saltos de línea (`NUEVA_LINEA`).
 - Detectar errores léxicos.
 3. Salida esperada:
 - Una lista ordenada de tokens que representen el programa de entrada.
-

Parte 2: Implementación de main

1. Realiza la implementación completa del archivo principal que muestra el flujo del sistema:
 - Se lee el pseudocódigo
 - Se ejecuta el lexer
 - Se imprimen los tokens

Requisitos técnicos

- Lenguaje de implementación: **Python**
- Modularidad obligatoria:
 - `Scanner` (lexer)
 - `TokenType`
 - `Token`
 - `main`
- Uso de buenas prácticas:
 - Tipos (`typing`)
 - Clases y encapsulamiento
 - Manejo de errores

Extensiones (no obligatorio)

- Soporte para funciones propias (`funcion`)
 - Manejo de indentación estilo Python (`INDENT` , `DEDENT`)
 - Soporte para colores personalizados y direcciones
-

Entregables

1. Código fuente completo a mano y también en texto plano.
2. Capturas de pantalla con ejemplos de entrada/salida.